JUNIOR TAPI HARUS SENIOR

Belajar Bertahan di Era AI

INDRA ZULFI MUSHODDAQ

Prologue

Aku akan coba kasih sedikit gambaran.

Tahun 2019, Kamu baru saja lulus dari perkuliahan computer science, dan pekerjaan pertama kamu menjadi developer adalah menulis kode, basic CRUD operations, benerin bugs kecil-kecil, dan perlahan kamu belajar dari codebase yang kamu pegang, sesekali kamu membuat kesalahan, Senior developers melakukan review dari kode yang kamu tulis, kemudian mengajarkan kamu mengapa seseorang yang melakukan nested-loop termasuk orang-orang yang ngeselin. :)

Dan hari ini 2025, Kamu lulus sekali lagi. Tapi kali ini sebuah AI (Artificial Intelligence) bisa menulis CRUD operations hanya dalam beberapa detik, bisa benerin bugs lebih cepet daripada kecepatanmu membaca error messages. Pekerjaan-pekerjaan yang harusnya bisa melatih kamu menjadi lebih baik? Yeap, sekarang sudah terotomasi bahkan sebelum kamu mencobanya.

Dan tahu apa yang paling absurd ? Pekerjaanmu sekarang

adalah melakukan review terhadap kerjaannya AI.

Kamu adalah seseorang yang harusnya belajar dari melakukan pekerjaan-pekerjaan yang *sederhana* tersebut, dan sekarang diekspektasikan untuk mampu mengevaluasi kode yang ditulis oleh AI yang dilatih dengan milyaran baris kode. Ini seperti remaja yang baru merantau, baru pertama kali nge-kos, baru pertama kali masak, terus diminta untuk mengevaluasi Masakan Chef Juna.

Inilah paradoksnya junior developer di masa AI seperti saat ini. Jika kamu merasakan hal ini saat ini, bingung dengan arah karir, frustasi, muncul ketakutan. Maka artinya kamu sedang memahami situasi ini, so kita akan coba untuk cari solusinya.

Understanding the Problem

Selama bertahun-tahun kebelakang, software development mengikuti predictable learning-curve. Junior developers mulai dengan apa yang biasa kita sebut ***"Training Wheels"***.

Mulai dengan :

1. Menulis kode boilerplate
2. Implementasi fitur yang simple
3. Consuming basic APIs
4. Benerin bugs yang cukup mudah.

Pekerjaan-pekerjaan ini bukan hanya sekedar pekerjaan yang membuatmu terlihat sibuk, Pekerjaan ini punya tujuan krusial, mereka membangun *engineering-intuition* para developer.

Ketika kamu mulai terbiasa menulis ratusan manual database query, kamu akan mulai merasakan *sesuatu* ketika ada yang salah. Ketika kamu melakukan debug exception yang rumit, kamu sebenarnya sedang mengembangkan indra ke-enam untuk *merasakan* dimana problem itu bersembunyi.

Dan AI datang menghapus training wheels ini.

Tools seperti Cursor, Claude Code, Trae, Opencode, Gemini atau yang semisalnya mampu membuat boilerplate dengan instant, mereka bisa melakukan konsumsi API, menulis database queries, dan membuat solution yang works hanya dalam beberapa detik.

Pekerjaan-pekerjaan simple yang harusnya menjadi pondasi pembelajaran para developer junior, sekarang sudah bisa dihandle oleh AI.

Ini membuat sesuatu yang disebut oleh para *Economist*: **"missing rungs"** problem. Dimana tangga karir telah dipotong, dan kamu tidak bisa naik mulai dari bawah dengan cara yang sama.

Dan Ironisnya ? Perusahaan-perusahaan yang memanfaatkan AI, masih butuh manusia untuk melakukan evaluasi dan verifikasi kode yang dihasilkan oleh AI.

AI mampu untuk menghasilkan *impressive-looking solutions* yang mungkin saja mengandung bugs yang tak terlihat, security vulnerabilities, atau bahkan architectural mismatches.

Yap, mereka butuh seseorang untuk menangkap problem-problem ini untuk tidak sampai memiliki impact negatif.

Dan ini terjadi bahkan di level junior developer. Padahal untuk melakukan review kode secara efektif, kamu membutuhan berbagai macam skill.

Kemampuan menerawang dimana error dan bugs terjadi, perlu kemampuan untuk tahu dimana terjadi ***subtle logical mistakes*** yang AI secara yakin mengatakan bahwa itu adalah solusi yang benar.

Memahami Architectural fit, pertanyaan yang harus selalu muncul seperti, *"apakah kode ini betul ditempatkan disini ?", "Apakah sudah mengikuti pattern dari sistem yang kita buat?"*

Kemampuan mengidentifikasi security vulnerabilites.

Kemampuan mengevaluasi performa dari kode, Apakah solusi ini scalable?

Skillset yang disebutkan di atas adalah “natural” skillnya para senior developer, yang mana membutuhkan pengalaman, pemahaman, konteks, kemampuan judgement yang tentu skill ini dikembangkan selama bertahun-tahun melalui berbagai macam pekerjaan dan latihan.

Ini seperti meminta seseorang untuk melakukan “proofread” sebuah novel dalam bahasa inggris, yang mana orang tersebut

masih belajar.

Pertanyaannya apakah mungkin ? Technically possible, maybe.

Apakah akan optimal ? Efisien ? Tentu saja tidak akan!

Sekarang coba mari kita berbicara jujur, kita tahu bahwasannya ***minimum-competency-bar*** untuk para developer junior telah naik cukup tinggi.

Sebelum adanya AI, memahami syntax dan basic pogramming concept merupakan bekal yang cukup untuk memulai, dan kamu belajar sisa-nya dari pekerjaan.

Sedangkan di hari ini para employers berekspektasi para juniors untuk punya pemahaman lain seperti :

- **System Architecture**
- **Security Awarenes**
- **Kemampuan meng-evaluasi kode**
- **Skill komunikasi yang efektif.**
- **Skill strategic problem-solving**

Yang aku rasa, ini *ga fair*. Ini seperti paradox seseorang yang membutuhkan SIM bahkan sebelum dia bisa duduk di dalam

mobil. But, unfortunately fairness ga sama sekali merubah kenyataan.

The Solution

Sebelum kita bicara skillset secara spesific, kita perlu untuk memahami sesuatu yang sangat fundamental. *How you think about your role?*

Mindset lama adalah mengukur kesuksesan dari output, seperti berapa banyak baris kode yang kamu tulis, berapa banyak tiket yang kamu selesaikan, seberapa cepat yang bisa shipping fitur.

Sedangkan *Mindset baru* yang harus kamu miliki adalah mengukur kesuksesan dari outcome.

Apakah solusi yang kamu offer benar-benar solving problem? Apakah kodebase yang kamu pegang akan menjadi lebih baik atau bahkan lebih buruk setelah kontribusi yang kamu lakukan? Apakah yang kamu lakukan sekarang bisa mencegah problem yang datang atau bahkan kamu sedang tidak sadar sedang membangun problem itu sendiri ?

Perubahan mindset ini membutuhkan sesuatu yang aku sebut dengan ***Three Strategic Questions***.

Sebelum kamu menulis kode apapun, atau bahkan mungkin sekarang sebelum kamu meminta AI melakukan sesuatu, kamu harus bertanya pada dirimu sendiri.

"What is the real problem ?"

Bukan pada surface-level dari deskripsi yang kamu baca pada tiket yang sedang kamu kerjaan, but the *actual problem*.

Seringkali sebuah tiket tertulis di Jira atau ClickUp adalah sebuah **"gejala"**, bukan penyebab. Seperti sebuah request untuk menambahkan "Add a loading spinner" mungkin saja adalah karena slow database query. Sebuah bug report mungkin memperlihatkan fundamental architectural flaw.

Junior developers yang mampu mengidentifikasi real problem akan secara exponensial lebih valuable dibandingkan seseorang yang *hanya* melakukan eksekusi sebuah perintah.

"What are the system-wide implications ?"

Setiap fitur ada dalam sebuah ekosistem internal, Sebuah

perubahan kecil terhadap sebuah module bisa merubah keseluruhan flow. Dan AI tidak bisa memahami batasan-batasan ini seperti Company constraint, team's convention, atau bahkan legacy system yang bahkan senior memilih izin sakit daripada harus menyentuhnya.

Kamu perlu berfikir jauh tentang system, bukan sekadar fitur.

"What problems am I creating for future developers?"

Kode itu lebih sering dibaca dibandingkan ditulis. Setiap keputusan yang kamu buat atau kamu setujui dari AI akan menurun kepada siapapun yang akan maintain kode ini berikutnya. Bisa jadi peer kamu, bisa jadi kamu sendiri yang setelah 6 bulan kamu lupa why things were done this way.

Setelah kamu memiliki mindset ini, sekarang kita coba lihat bagaimana practical framework aku ketika mengevaluasi kode yang di generate oleh AI. Mungkin kamu bisa anggap ini sebagai pre-flight checklist yang kamu tidak akan pernah mau untuk melewatkan setiap stepnya.

Checkpoint 1 : Architectural Fit.

The question : “Does this code belong here?”

Setiap codebase memiliki patterns, bisa jadi MVC, bisa saja microservices, atau bahkan architectural “unik” yang dihasilkan oleh seorang developer yang dikejar deadline jam 3 pagi 5 tahun yang lalu. Have you feel this ? Haha.

Whatever it is. AI tidak secara otomatis memahami atau bahkan mengikuti pattern ini.

The action : Sebelum kamu tekan tombol accept AI Code. coba untuk pahami dan identifikasi design pattern dari module yang sedang kamu kerjakan. Apakah kode yang baru ini mengikuti pattern yang sama ? Apakah kode ini mengikuti boundary yang telah ada ?

Seperti contoh ketika kamu menggunakan ORM untuk database access dan tetiba AI menulis RAW SQL, thats a flag. Tidak selalu salah, but definitely worth questioning.

Checkpoint 2 : Security Review.

The question : "Is this code opening doors that should stay closed ?”

AI seringkali memprioritaskan functionality diatas security. AI terus “memaksa” untuk make things works, yang less concerned apakah kode ini safe atau tidak.

The action : Check stiap kode yang menerima user input atau eksternal system. Cari common vulnerabilities, seperti SQL injection, XSS, hardcoded credentials, improper authentication checks.

Tips: Tanyakan AI sekaligus, contoh prompt seperti “Review this code for OWASP top 10 vulnerabilites” Mungkin tidak akan secara sempurna menangkap seluruh problem. Tapi ini bisa menjadi starting point.

Checkpoint 3 : Performance Analysis.

The question : "Will this solution scale ?”

AI seringkali memberikan elegant-solution yang mungkin works pada small datasets tapi *hancur* ketika berhadapan dengan database yang ada di production.

Kadang pada satu solusi, mungkin saja AI melakukan $O(n^2)$ bahkan ketika $O(n)$ sangat memungkinkan. Bisa jadi satu database query menghasilkan N+1 problem yang hampir tidak terlihat oleh junior developers.

The action : Segala kode yang mengandung loops, database calls, atau bahkan data structures, Pastikan untuk melakukan tracing dari time-complexity-nya. Selalu tanyakan pada dirimu sendiri, “Kalo seandainya kita punya 10x users, atau bahkan 100x lebih banyak data, apakah solusi ini masih works?”

Checkpoint 4 : Readability Assestment.

The question : "Can humans understand this ?”

Baris kode adalah bagian dari komunikasi, Jika kamu tidak mampu menjelaskan sebuah fungsi pada kolega kamu yang non-technical, bisa jadi karena fungsi itu terlalu kompleks.

The action : Baca semua AI Generated Code. Baca dengan keras jika perlu, jika kamu merasa kebingungan ketika sedang menjelaskan kode ini, *that’s a sign of a problem*. Cek kembali apakah nama variablenya cukup meaningful, apakah fungsi-fungsi yang ditulis punya clear purpose, dan sesuatu yang non-obvius harus memiliki explanatory comments.

Checkpoint 5 : Testing verification.

The question : "How do we know this works?”

AI-generated tests seringkali superficial. Test yang digenerate seringkali adalah the happy-path, tanpa memperdulikan edge-cases.

The action : Jangan pernah percaya dengan AI-Generated test secara implisit, setidaknya tulis satu buah edge-case test dengan tanganmu, sesuatu yang mungkin dilewatkan AI, pastikan bahwa error-path di tangani dengan baik.

Sekarang kita akan coba untuk memahami metode ***"the problem framing drill"***. Latihan practical yang bisa aku sarankan dan framework ini akan merubah bagaimana kamu melakukan development.

Let say, sebuah scenario.

Kamu diminta untuk mengerjakan fitur profile picture upload.

Traditional Junior Approach :

Focus : How do i write the file upload code ?

Questions : What's the API endpoint ? What library handle file uploads?

Strategic Approach :

Sebelum kamu menulis atau melakukan prompting apapun, jawab dulu pertanyaan-pertanyaan ini :

Business/Product Layer :

- Kenapa user butuh profile pic? Personalization ? Trust ? Legal Requirements ?
- Berapa maksimal ukuran dari file ? *Ini akan mempengaruhi infrastruktur kamu.
- Format apa yang diperbolehkan ? PNG ? WebP ? Bagaimana jika user mengupload HEIC dari iPhone?

Architectural Layer

- Dimana images akan disimpan ? S3 ? Bagaimana dengan CDN?
- Haruskah upload file ini jadi bagian dari sebuah service secara monolith? atau perlu membuat service yang baru ?

Security Layer

- Bagaimana cara mencegah malicious uploads? Seperti mengupload executable file atau embedded scripts?
- Bagaimana kita melakukan image processing tanpa harus membuat vulnerabilities ?
- Siapa yang diizinkan untuk melihat user profiles pictures ? Apakah akan perlu untuk berhati-hati terhadap PII ?

Performance Layer

- Haruskah upload ini synchronous ? atau async ?
- Bagaimana cara serving images ini kepada ratusan ribu users tanpa mengorbankan performa server ?

Dokumentasikan setiap jawaban kamu bahkan sebelum kamu menulis kode. Ini akan membiasakan atau bahkan memaksa kamu untuk terbiasa melakukan strategic thinking. Ini membangun kemampuan judgement kamu yang skill ini tidak pernah tergantikan oleh AI.

Pelajari juga System Design atau Architectural Solution. Dan tips jika kamu ingin belajar *architectural intution* lebih cepat, Maka aku sarankan kamu untuk belajar system architecture dari aplikasi yang sudah biasa kamu gunakan, apapun itu.

Google Drive, URL shortener, Whatsapp, Telegram, Youtube, apapun yang biasa kamu gunakan, dan lakukan dekonstruksi.

- Gambar bagian-bagian komponennya, Load balancer, Web Server, Database, Cache Layer, Message Queus.
- Trace dari data flownya. Ketika user membuat sebuah tweet, apa yang terjadi ? Kemana data akan dilanjutkan ? System apa saja yang menangani ini ?
- Identify the patterns. Kenapa mereka memilih teknologi ini
- Problem apa yang mereka sedang solve.
- Apply to your work. Ketika kamu membuat logging feature. Tanyakan pada dirimu "Bagaimana twitter melakukan handling ini? Apakah langsung masuk ke dalam database ? atau mungkin dihandle oleh message queue?"

Latihan-latihan ini akan membuat kamu memiliki *mental-library of Architectural Solution*.

The Human Skills AI Can't Replace

Technical skills sangat penting, tetapi Developer yang thriving di era AI seringkali memiliki kapabilitas yang lebih baik.

Critical Thinking

AI sangat-sangat confident, bahkan terkadang sangat over-confident. Seringkali mempresentasikan incorrect-solutions seakan yang paling baik.

Tugas kamu adalah mempertanyakan segala sesuatu, challenge asumsinya, biasakan untuk mempertimbangan solusi alternatif, Dan jangan pernah lelah bertanya “Kenapa begini” ?

Communication

Biasakan untuk melatih diri kamu menjelaskan satu konsep teknis kepada orang-orang yang tidak memiliki pemahaman teknis. Ini mampu melatih kamu untuk terbiasa mengartikulasikan alasan terhadap solusi yang kamu buat. Biasakan juga untuk memahami cara menerjemahkan business requirements kepada technical implementation.

Adaptability

Tools yang kamu pakai untuk belajar hari ini bisa saja akan menjadi usang dalam beberapa tahun, framework yang biasa kamu pakai akana berubah, best practice akan berubah. perubahan itu inevitable.

Junior Vs Senior, What actually differentiate junior from senior these days ?

Mungkin kamu memiliki pertanyaan, jika para junior melakukan hal ini, lalu apa yang membedakan junior dan senior di era ini ? Keduanya sama-sama melakukan strategic problem-solving dan AI code review, apa yang membedakan ?

So, sebenarnya bukan pada what they do,

It's about ***scope, context and autonomy***.

Scope of Impact :

Junior fokus pada satu fitur atau module. Strategic decision mereka memiliki local-effects. Yang mungkin saja terjadi bug atau technical debt, tetapi pengaruhnya pada lingkup module atau fitur tersebut.

Senior fokus pada system secara keseluruhan, tim, dan product lines. Decision yang mereka buat bisa berpengaruh pada organisasi, poor-architectural choice akan berpengaruh kepada bisnis selama bertahun-tahun kedepan.

Depth of Context :

Junior memiliki cukup konteks untuk melakukan validasi kode terhadap requirements. “Apakah ini berfungsi sebagaimana yang diminta di dalam tiket?”

Sedangkan senior punya cross-functional context. Mereka memahami business implications, security landscape dan infrastructure constaint. Mereka melakukan validasi kode terhadap long-term strategy dan organizational risk.

Level of Autonomy :

Junior akan bekerja dengan arahan, mereka membuat proposed-solutions dan menerima feedback. Kemampuan junior dalam melakukan judgement akan berkembang.

Sedangkan senior bekerja secara independent dan melakukan mentoring terhadap yang lain. Judgement mereka dipercaya, dan mereka yang memberikan feedback.

Yang terakhir :

So dengan keadaan yang saat ini terjadi, dimana kamu akan menempatkan dirimu ?

Aku bisa katakan bahwasannya pekerjaan sebagai junior developer tidak benar-benar mati karena AI. Role ini hanya sedang berubah. Tugasnya berubah, Ekpektasinya berubah, tapi kebutuhan fundamentalnya tetap sama. Seseorang yang mampu untuk memahami problem, kemudian membuat solusi dan melakukan implementasi kepada software.

Value kamu meningkat dari berbagai macam arah :

- **Contextual Expertise.**
- **Judgement**

- **Strategic Thinking**
- **Human Connection.**

Developers yang sedang struggling adalah yang melihat AI sedang melakukan pekerjaan mereka.

Sejujurnya Aku tidak ingin melakukan sugarcoat.

Yes, the path is harder now, the bar even higher.

Traditional path telah berubah.

Tapi satu hal yang aku percaya bahwa. Dvelopers yang beradaptasi dngan realitas baru ini bahkan akan lebih baik dari previous generation. Bukan karena mereka lebih pintar, atau bahkan lebih berbakat. Tapi karena mereka “dipaksa” untuk memiliki kemampuan strategic thinking yang lebih cepat. Mereka sedang membangun kemampuan judgement bahkan mulai dari hari pertama. Bukan tahun ke 4.

Skillset ini kamu dapatkan dari secara kritis melakukan review pada AI output, berfikir secara sistematis, bertanya “kenapa” sebelum “bagaimana”. Dan skillset ini compunding tiap waktunya.

Take a breath.

Be positive. Beradaptasilah terhadap perubahan.

Aku yakin kamu bisa. Keep grinding!

Thank you for reading!

About the Author

Indra Zulfi Mushoddaq, Pragmatic Software and AI Engineer, Founder & Mentor Devscale Indonesia, devscale.id